

LAB 3

Objective

Evaluate different architectural styles, select the most appropriate one for the assigned scenario, and create a comprehensive UML Component Diagram showing modules, interfaces, and dependencies for the Alumni Mentorship & Mock Interview Platform.

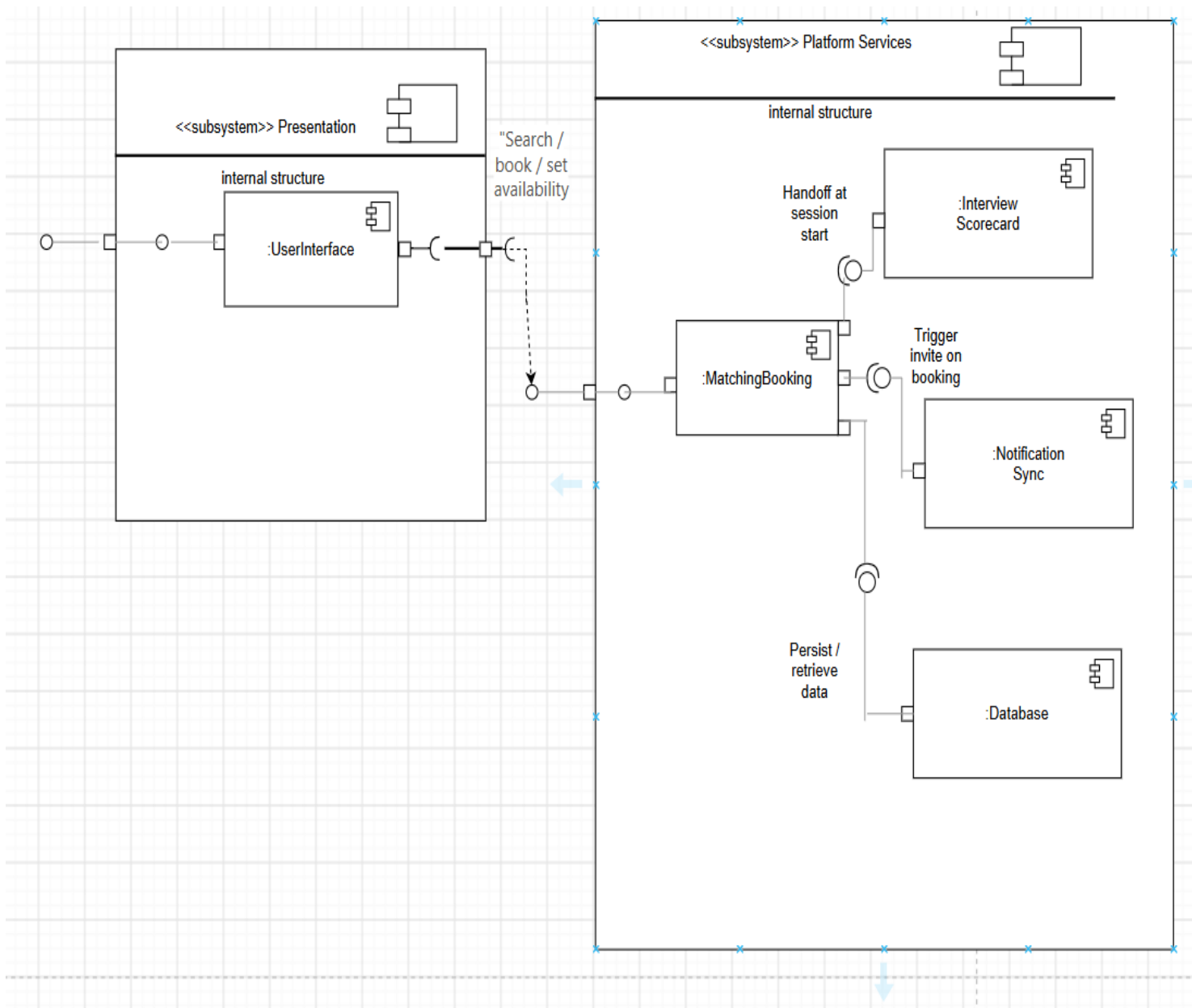
5 Components

1. User Interface Component — student and mentor web/app interactions
2. Matching & Booking Component — domain-based mentor matching, session booking, availability management
3. Interview & Scorecard Component — running the mock interview session and recording the scorecard
4. Notification & Calendar Sync Component — sends calendar invites and reminders
5. Database Component — stores profiles, availability, sessions, and scorecards; enforces soft-deletion

4 Interfaces

1. User Interface ↔ Matching & Booking — Search / book / set availability
2. Matching & Booking ↔ Interview & Scorecard — Handoff at session start
3. Matching & Booking ↔ Notification & Calendar Sync — Trigger invite on booking
4. Matching & Booking ↔ Database — Persist / retrieve data

COMPONENT DIAGRAM



Architectural Justification

Alumni Mentorship & Mock Interview Platform

Architecture Selection: We chose **Layered Architecture** for the Alumni Mentorship & Mock Interview Platform.

Reason 1: Clear Separation of Concerns Matches the Workflow

The platform's core flow is naturally sequential: a student interacts with the interface, the business logic matches and books mentors, and the outcome is stored for later retrieval. A Layered Architecture (Presentation, Business, Data) mirrors this request flow directly. The User Interface Component stays purely presentational, the Matching & Booking Component (along with Interview & Scorecard and Notification & Calendar Sync) encapsulates all business rules, and the Database Component handles only persistence. This keeps each layer easy to reason about, test, and modify independently — important for a student-built system with limited development time.

Reason 2: Centralized Orchestration Avoids Duplicated Logic

Since the Matching & Booking Component acts as the central orchestrator for domain matching, slot locking, scorecard handoff, and notification triggers, a Layered Architecture lets this business logic live in one place. A Microservices approach would force this same logic to be duplicated or coordinated across independently deployed services — unnecessary operational complexity for a single-institution platform with a moderate, predictable user base (students and alumni of one university).

Security Advantage

In this architecture, only the Business Layer is permitted to communicate directly with the Database Component; the Presentation Layer never queries the database itself. This enforces a single, centralized checkpoint where authentication, authorization, and input validation happen before any read or write — for example, verifying a student's identity before allowing a scorecard to be written, or enforcing the soft-deletion rules required for user privacy (NFR-001). Consolidating data access through one layer significantly reduces the attack surface compared to an architecture where the UI could query the database directly.

Performance Benefit

Because all mentor availability and booking data is concentrated behind a single Business Layer boundary, the system can introduce a caching layer at that one point — for example, caching the results of “Search Mentors by Domain” queries. This is especially valuable during predictable peak periods, such as the start of a semester when many students book mentorship sessions at once. Centralizing the cache at a single layer avoids the added latency and cache-invalidation complexity that would come from coordinating caching across multiple independently-scaled microservices.